Ex. No. 1	WORKING WITH WIDGETS AND LAYOUTS
Date of Exercise	31/08/2026

Aim

To develop a Flutter application using basic widgets and layout widgets such as Container, Row, Column, Expanded, Card, and ListView.

Description

This project focuses on designing and creating a clean, structured visual user interface for a Student Profile Dashboard utilizing the Flutter framework. The architecture of the application features nested architectural UI blocks:

- **Application Scaffold:** Forms the core structure with a designated AppBar named "Student Profile".
- **ListView Container:** Wraps the primary viewport layout block to provide a flexible scrolling capability across different hardware display heights, entirely preventing overflow errors.
- **Decorated Container:** Houses institutional text tracks with specified margin offsets and inner structural paddings.
- **Column & Row Components:** Manages hierarchical alignment, handling sequential text stacks vertically and functional system navigation icons horizontally with even spatial layouts.
- **Material Cards & Lists:** Incorporates specialized Material layout modules (Card, ListTile) featuring visual asset indicators (Icons) to neatly organize course information.

Procedure:

1. Verify SDK Setup: Launch your local system terminal tool and execute flutter doctor to ensure the core SDK installation is clear. Use flutter devices to list out targeted runtime environments

2. Initialize Project Environment: Snin up a pristine layout template architecture by calling the command line option: flutter create widget_layout.

3. Workspace Redirection: Jump inside the root level workspace folder using cd widget_layout and initiate the visual workspace interface by calling code ..

4. Target Code Modifications: Locate and load the file target pathway lib/main.dart, clean out the boilerplate system generation code snippets entirely, and map in the declarative Material entry configurations.

5. Build Column & Container Hierarchies: Set up layout structures by integrating a vertical layout stack (Column), wrapping individual child modules with customizable layout sheets (Container) that contain text strings like "Raj" and "CSE".

6. Organize Horizon Components: Embed horizontal alignments using a specialized Row with spatial arrangements set to MainAxisAlignment.spaceEvenly to distribute structural context layout icons smooth

7. Generate Dynamic Scrolling Sheets: Incorporate multiple item layouts by using an extensible scrollable view (istView) combined with individual nested list components (ListTile) inside physical design wrappers (Card).

8. Compilation & Testing: Fire up your targeted development emulator tool and execute the local system task script flutter run inside the terminal to audit structural rendering.

Program

```
import 'package:flutter/material.dart';


void main() {
  runApp(const MyApp());
}


class MyApp extends StatelessWidget {
  const MyApp({super.key});


  @override
  Widget build(BuildContext context) {
    return MaterialApp(
    debugShowCheckedModeBanner: false,
    home: Scaffold(
        appBar: AppBar(
          title: const Text('Student Profile'),
          backgroundColor: Colors.blue,
          foregroundColor: Colors.white,
        ),
        body: ListView(
          padding: const EdgeInsets.all(10),
          children: [
```

```
// Container with embedded Column Layout
Container(
  padding: const EdgeInsets.all(20),
  margin: const EdgeInsets.all(10),
  decoration: BoxDecoration(
  color: Colors.blue.shade50,
    borderRadius: BorderRadius.circular(10),
    border: Border.all(color: Colors.blue.shade100),
  ),
  child: const Column(
    children: [
      Text(
        'Student Profile',
        style: TextStyle(fontSize: 20, fontWeight: FontWeight.bold),
      ),
      SizedBox(height: 10),
      Text('DICKSON S', style: TextStyle(fontSize: 18)),
      Text('CSE', style: TextStyle(fontSize: 16, color: Colors.grey)),
      SizedBox(height: 5),
      Text('Computer Science and Engineering'),
      Text('Flutter Laboratory'),
    ],
```

```
  ),
),

const SizedBox(height: 15),

// Row Layout displaying horizontal icons
Row(
  mainAxisAlignment: MainAxisAlignment.spaceEvenly,
  children: const [
    Icon(Icons.home, size: 30, color: Colors.blue),
    Icon(Icons.person, size: 30, color: Colors.blue),
    Icon(Icons.settings, size: 30, color: Colors.blue),
  ],
),

const SizedBox(height: 20),

// Card Layout with wrapped ListTile elements
const Card(
  elevation: 4,
  margin: EdgeInsets.symmetric(horizontal: 10, vertical: 5),
  child: ListTile(
    leading: Icon(Icons.book, color: Colors.orange),
    title: Text('Flutter'),
```

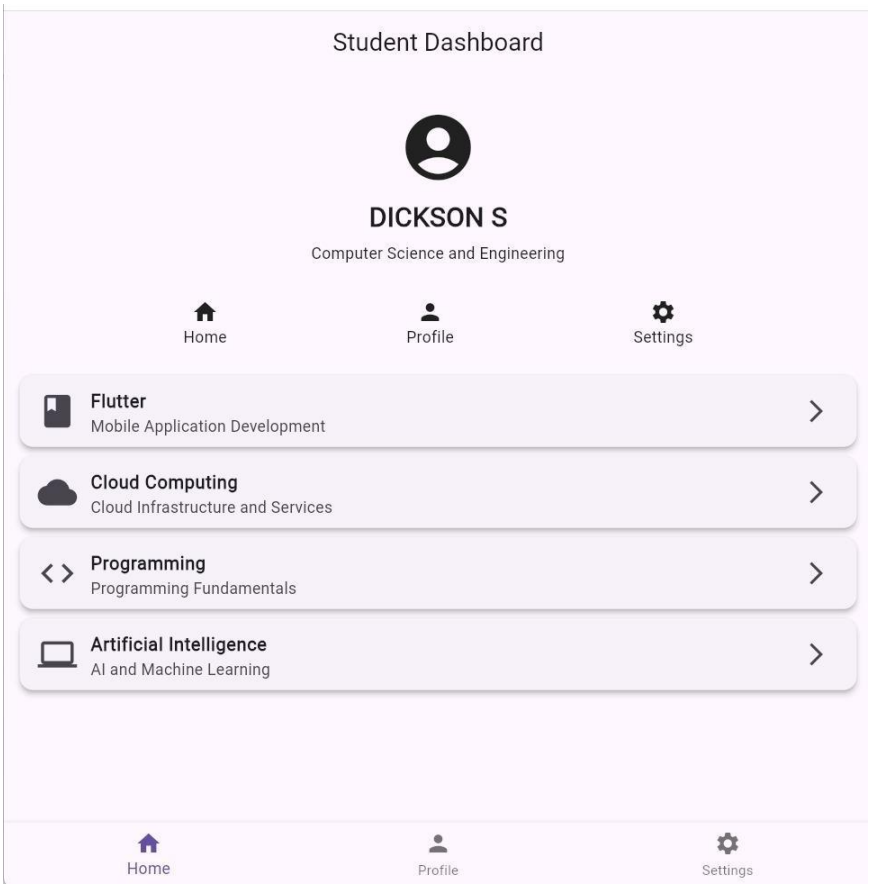
```
    subtitle: Text('Mobile Application Development'),
  ),
),

// ListView item sets mapping out supplementary courses
const Card(
  elevation: 2,
  margin: EdgeInsets.symmetric(horizontal: 10, vertical: 5),
  child: ListTile(
    leading: Icon(Icons.cloud, color: Colors.indigo),
    title: Text('Cloud Computing'),
    subtitle: Text('Distributed Architecture'),
  ),
),

const Card(
  elevation: 2,
  margin: EdgeInsets.symmetric(horizontal: 10, vertical: 5),
  child: ListTile(
    leading: Icon(Icons.code, color: Colors.green),
    title: Text('Programming'),
    subtitle: Text('Core Engineering Development'),
  ),
),
```

```
        ],
    );
  }
}
```

Output



Result:

The Flutter student profile and dashboard user interface application was successfully developed, structured, and executed using standard layout widgets, satisfying all design requirements without layout compilation constraint errors.